

Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій

ЗВІТ

з проектної роботи

з дисципліни “ Паралельні та розподільні обчислення ”

на тему “ Паралельні обчислення з використанням технології OpenMP”

Виконала:

Студент 2 курсу

групи ФЕП-23

Бойчук Карина

Мета: Реалізувати паралельний LU-розклад для квадратної матриці. Використовуючи розподіл даних між процесами. Очікувані результати: Оцінка прискорення при використанні різної кількості потоків. Порівняння часу виконання із стандартним послідовним LU-розкладом.

Хід роботи:

Теоретичні відомості

Паралельні обчислення

Паралельні обчислення — це метод виконання обчислювальних задач, при якому декілька операцій виконуються одночасно на різних обчислювальних ресурсах. Основною метою паралельних обчислень є підвищення швидкодії програм та ефективне використання апаратних ресурсів комп'ютера.

У сучасних комп'ютерах широко використовуються багатоядерні процесори, що дозволяє виконувати кілька потоків виконання одночасно. Для ефективного використання таких можливостей застосовуються спеціальні технології паралельного програмування.

Однією з найбільш поширених технологій для створення паралельних програм на багатоядерних процесорах є OpenMP (Open Multi-Processing). Ця технологія дозволяє розпаралелювати виконання програм за допомогою спеціальних директив компілятора, бібліотечних функцій та змінних середовища.

Паралельні обчислення дозволяють:

- значно скоротити час виконання складних обчислень;
- ефективно використовувати ресурси багатоядерних процесорів;
- обробляти великі обсяги даних;
- підвищувати продуктивність наукових та інженерних обчислень.

У задачах лінійної алгебри, таких як обчислення LU-розкладу матриці, паралельні обчислення дозволяють значно прискорити виконання алгоритмів, особливо при роботі з великими матрицями.

Технологія OpenMP

OpenMP — це стандарт програмування для створення паралельних програм у мовах C, C++ та Fortran. Він призначений для систем із спільною пам'яттю та дозволяє легко реалізовувати багатопотокові програми.

Основна ідея OpenMP полягає у використанні директив компілятора, які додаються до звичайної послідовної програми та дозволяють автоматично створювати паралельні області виконання.

Основні переваги OpenMP:

- простота використання;
- можливість поступової паралелізації програм;
- підтримка більшістю сучасних компіляторів;
- ефективна робота з багатоядерними процесорами.

Завдяки цим перевагам OpenMP широко використовується у наукових обчисленнях, інженерних задачах, обробці великих масивів даних та моделюванні складних систем.

Основні директиви OpenMP:

Директива parallel:

```
#pragma omp parallel
```

створює паралельну область, у якій виконання програми розділяється між кількома потоками. Кожен потік виконує одну і ту ж частину коду, але може працювати з різними даними.

Директива single:

```
#pragma omp single
```

вказує, що певний блок коду повинен виконуватися лише одним потоком із усієї команди потоків. Інші потоки в цей час очікують завершення виконання цього блоку. Ця директива часто використовується для створення задач або ініціалізації даних.

Паралельні цикли в OpenMP

Для паралельного виконання циклів використовується директива:

```
#pragma omp for
```

Вона розподіляє ітерації циклу між потоками.

Вимірювання часу виконання

Для вимірювання часу виконання паралельних програм в OpenMP використовується функція:

```
omp_get_wtime()
```

Вона повертає поточний час у секундах.

Завдання (Tasks) в OpenMP

Однією з важливих можливостей OpenMP є технологія задач (tasks).

Задача — це незалежна частина роботи, яка може виконуватися будь-яким потоком.

Для створення задачі використовується директива:

```
#pragma omp task
```

Синхронізація задач

Для очікування завершення всіх задач використовується директива:

```
#pragma omp taskwait
```

Вона блокує виконання потоку до завершення всіх створених задач.

LU-розклад матриці

LU-розклад — це метод розкладання квадратної матриці на добуток двох матриць:

$$A=L \times U$$

де:

- L — нижня трикутна матриця
- U — верхня трикутна матриця

Цей метод широко використовується для:

- розв'язування систем лінійних рівнянь;
- обчислення обернених матриць;
- знаходження визначника матриці.

У класичному алгоритмі LU-розкладу обчислення виконуються послідовно. Однак для великих матриць це може займати значний час.

Паралельний LU-розклад

Паралельна реалізація LU-розкладу дозволяє розподілити обчислення між кількома потоками.

Основна ідея полягає у тому, що обчислення елементів матриць L та U можуть виконуватися незалежно для різних рядків або стовпців.

У паралельній реалізації використовуються задачі OpenMP, які створюють незалежні обчислення для кожного елемента або групи елементів матриці.

Завдяки цьому:

- різні частини алгоритму виконуються одночасно;
- зменшується загальний час обчислень;
- підвищується продуктивність програми.

Ефективність паралельного алгоритму оцінюється шляхом порівняння часу виконання з послідовною реалізацією.

Покроковий алгоритм роботи програми:

Спочатку програма запускається і запитує у користувача розмір матриці n . Якщо введене значення перевищує максимально допустиме (1000), програма завершує роботу з повідомленням про помилку. Далі користувач обирає спосіб заповнення матриці A : ручне введення, випадкова генерація або генерація за формулою. В залежності від вибору викликається відповідна функція, яка заповнює матрицю A .

Після цього матриця A виводиться на екран. Далі програма переходить до обчислення LU-розкладу послідовним методом. Спочатку матриці L і U обнуляються. Потім фіксується час початку виконання і викликається функція послідовного LU-розкладу. У цій функції для кожного рядка і спочатку обчислюються елементи рядка матриці U , використовуючи вже обчислені значення L і U , а потім обчислюються елементи стовпця матриці L . На діагоналі матриці L встановлюються одиниці. Після завершення обчислень фіксується час завершення і визначається тривалість виконання послідовного алгоритму, яка виводиться на екран.

Далі програма переходить до паралельних обчислень. Задається набір кількостей потоків (1, 2, 4, 8), і для кожного значення виконується окремий запуск паралельного алгоритму. Перед кожним запуском встановлюється відповідна кількість потоків, матриці L і U знову обнуляються, і фіксується час початку. У паралельній функції створюється паралельна область OpenMP, де один потік організовує створення задач. Для кожного кроку і створюються задачі для обчислення елементів рядка матриці U , після чого виконується очікування завершення цих задач. Потім створюються задачі для обчислення елементів стовпця матриці L , і знову відбувається очікування їх завершення. Після виконання всіх ітерацій паралельна область завершується. Далі фіксується час завершення і обчислюється тривалість виконання для кожної кількості потоків, яка виводиться на екран.

Після виконання всіх експериментів результати записуються у файл result.txt. У файл заноситься розмір матриці, час послідовного виконання, часи паралельного виконання для різної кількості потоків, а також отримані матриці L і U . Після цього програма виводить повідомлення про успішне збереження результатів і завершує роботу.

Приклад прокоментованого коду:

```
C lu_openmp.c > reset_LU(int)
1  #include <stdio.h>    // стандартний ввід/вивід (printf, scanf)
2  #include <stdlib.h>   // для rand(), srand()
3  #include <omp.h>      // бібліотека для паралельних обчислень OpenMP
4  #include <time.h>     // для генерації випадкових чисел через час
5
6  #define MAX 1000      // максимальний розмір матриці
7
8  // глобальні матриці (щоб не передавати їх у кожен функцію)
9  double A[MAX][MAX];  // початкова матриця
10 double L[MAX][MAX];  // нижня трикутна
11 double U[MAX][MAX];  // верхня трикутна
12
13 // очищення матриць L і U
14 void reset_LU(int n)
15 {
16     // просто проходимося по всіх елементах і зануляємо
17     for(int i=0;i<n;i++)
18         for(int j=0;j<n;j++)
19             {
20                 L[i][j] = 0.0;
21                 U[i][j] = 0.0;
22             }
23 }
24
25 // введення матриці вручну
26 void input_matrix_console(int n)
27 {
28     printf("\nEnter matrix elements:\n");
29
30     // користувач сам вводить кожен елемент
31     for(int i=0;i<n;i++)
32         for(int j=0;j<n;j++)
33             {
34                 printf("A[%d][%d] = ",i,j);
35                 scanf("%lf",&A[i][j]);
36             }
37 }
```

Забезпечено введення вхідних даних вручну, автоматично за допомогою випадкової генерації, а також за допомогою довільної функції:

```

24
25 // введення матриці вручну
26 void input_matrix_console(int n)
27 {
28     printf("\nEnter matrix elements:\n");
29
30     // користувач сам вводить кожен елемент
31     for(int i=0;i<n;i++)
32         for(int j=0;j<n;j++)
33         {
34             printf("A[%d][%d] = ",i,j);
35             scanf("%lf",&A[i][j]);
36         }
37 }
38
39 // випадкова генерація
40 void generate_matrix_random(int n)
41 {
42     srand(time(NULL)); // ініціалізація генератора
43     // заповнюємо числами від 1 до 10
44     for(int i=0;i<n;i++)
45         for(int j=0;j<n;j++)
46             A[i][j] = rand()%10 + 1;
47 }
48
49 // генерація за функцією
50 void generate_matrix_function(int n)
51 {
52     // проста формула щоб було легко перевірити правильність
53     for(int i=0;i<n;i++)
54         for(int j=0;j<n;j++)
55             A[i][j] = i + j + 1;
56 }
57

```

Забезпечено виведення результатів роботи у файл:

```

// запис результатів у файл
void write_result(int n, double seq_time, double times[], int threads_count)
{
    FILE *f = fopen("result.txt","w"); // відкриваємо файл

    fprintf(f,"Matrix size: %d\n\n",n);

    // записуємо час послідовного алгоритму
    fprintf(f,"Sequential time: %f\n\n",seq_time);

    fprintf(f,"Parallel results:\n");

    int thread_list[4] = {1,2,4,8};

    // запис часу для різної кількості потоків

```

```

for(int i=0;i<threads_count;i++)
    fprintf(f,"Threads %d : %f seconds\n",thread_list[i],times[i]);

// запис матриці L
fprintf(f,"\nMatrix L:\n");

for(int i=0;i<n;i++)
{
    for(int j=0;j<n;j++)
        fprintf(f,"%8.3f ",L[i][j]);

    fprintf(f,"\n");
}

// запис матриці U
fprintf(f,"\nMatrix U:\n");

for(int i=0;i<n;i++)
{
    for(int j=0;j<n;j++)
        fprintf(f,"%8.3f ",U[i][j]);

    fprintf(f,"\n");
}

fclose(f); // закриваємо файл

```

Проект включає у собі реалізацію тем, вивчених під час лабораторних робіт 1-4, окрім цього, загальною вимогою є використання технології завдань (#pragma omp task) де це можливо:

```

// паралельний LU з використанням tasks
void lu_parallel(int n)
{
#pragma omp parallel    // запускаємо паралельну область
{
#pragma omp single      // тільки один потік створює задачі

    for(int i=0;i<n;i++)
    {

        // ----- U -----
        for(int k=i;k<n;k++)
        {

#pragma omp task firstprivate(i,k) // створюємо задачу
        {
            double sum = 0;

            for(int j=0;j<i;j++)
                sum += L[i][j]*U[j][k];

```

```

        U[i][k] = A[i][k] - sum;
    }
}

#pragma omp taskwait // чекаємо завершення задач

    // ----- L -----
    for(int k=i;k<n;k++)
    {
#pragma omp task firstprivate(i,k)
        {
            if(i==k)
                L[i][i] = 1;
            else
            {
                double sum = 0;

                for(int j=0;j<i;j++)
                    sum += L[k][j]*U[j][i];

                L[k][i] = (A[k][i] - sum)/U[i][i];
            }
        }
    }

#pragma omp taskwait // знову чекаємо

    }
}
}

```

Наявність чітких результатів роботи програми проєкту:

```
PS D:\паралельні\project1> ./lu
Enter matrix size: 3

Choose input method:
1 - Manual input
2 - Random generation
3 - Function generation
Choice: 2

Matrix A:
  1.00   7.00   3.00
  6.00   9.00   3.00
  2.00   3.00   9.00

Sequential time: 0.000000
Threads 1 : 0.000000
Threads 2 : 0.001000
Threads 4 : 0.001000
Threads 8 : 0.000000

Results saved to result.txt
PS D:\паралельні\project1>
```

```
result.txt
Файл  Редагувати  Переглянути

Matrix size: 3

Sequential time: 0.000000

Parallel results:
Threads 1 : 0.000000 seconds
Threads 2 : 0.001000 seconds
Threads 4 : 0.001000 seconds
Threads 8 : 0.000000 seconds

Matrix L:
  1.000   0.000   0.000
  6.000   1.000   0.000
  2.000   0.333   1.000

Matrix U:
  1.000   7.000   3.000
  0.000 -33.000 -15.000
  0.000   0.000   8.000
```

Порівняння досягнутих результатів з їх послідовними реалізаціями, а також між собою за різної кількості потоків та розміру даних:

```
Sequential time: 0.210000
Threads 1 : 0.232000
Threads 2 : 0.223000
Threads 4 : 0.319000
Threads 8 : 1.012000
```

```
Sequential time: 1.695000
Threads 1 : 1.768000
Threads 2 : 1.175000
Threads 4 : 1.238000
Threads 8 : 3.854000
```

У ході виконання експериментів було проведено дослідження ефективності паралельної реалізації алгоритму для двох розмірів вхідних даних: матриць розміром 500×500 та 1000×1000 . Для кожного випадку здійснювалося порівняння часу виконання послідовної реалізації та паралельної реалізації з використанням 1, 2, 4 і 8 потоків.

Для матриці розміром 500×500 послідовний час виконання становив 1.695 секунди. При використанні одного потоку час виконання збільшився до 1.768 секунди, що пояснюється накладними витратами на створення та керування потоками в OpenMP, які не компенсуються обсягом обчислень. При використанні двох потоків спостерігається найкращий результат — час виконання зменшується до 1.175 секунди, що свідчить про досягнення прискорення порівняно з послідовною реалізацією. Однак при подальшому збільшенні кількості потоків до чотирьох ефективність дещо знижується, і час виконання становить 1.238 секунди. Найгірший результат спостерігається при використанні восьми потоків — 3.854 секунди, що значно перевищує навіть

послідовний час. Це пояснюється значними накладними витратами, конкуренцією за ресурси процесора та пам'яті, а також можливими проблемами синхронізації потоків. Таким чином, для матриці середнього розміру оптимальною є невелика кількість потоків, зокрема два.

Для матриці розміром 1000×1000 послідовний час виконання становив 0.210 секунди, що є значно меншим, ніж у попередньому випадку (ймовірно через особливості реалізації або умов експерименту). При використанні одного потоку час виконання зріс до 0.232 секунди, а при двох потоках — до 0.223 секунди. У цих випадках паралельна реалізація не демонструє суттєвого покращення і навіть дещо поступається послідовній. При використанні чотирьох потоків час виконання збільшується до 0.319 секунди, а при восьми потоках — до 1.012 секунди, що вказує на значне погіршення продуктивності. У даному випадку накладні витрати на організацію паралельних обчислень перевищують вигаиш від розпаралелювання, оскільки сам алгоритм виконується досить швидко навіть у послідовному режимі.

Порівнюючи обидва дослідження, можна зробити висновок, що ефективність паралельних обчислень суттєво залежить від розміру задачі. Для матриці 500×500 спостерігається реальне прискорення при використанні двох потоків, тоді як для матриці 1000×1000 паралелізація не дає позитивного ефекту. Крім того, в обох випадках збільшення кількості потоків до восьми призводить до значного зниження продуктивності. Це свідчить про те, що надмірна кількість потоків може негативно впливати на швидкодію через перевантаження процесора, збільшення витрат на синхронізацію та неефективне використання кеш-пам'яті.

Отже, результати дослідження показують, що паралельна реалізація є доцільною лише за певних умов, зокрема при достатньо великому обсязі обчислень і правильно підібраній кількості потоків. Оптимальна кількість потоків у даних експериментах становить 2, тоді як використання більшої кількості потоків є недоцільним і призводить до погіршення результатів.

Висновок: У даній лабораторній роботі було розглянуто теоретичні основи LU-розкладу матриць, зокрема метод Дулітла, який дозволяє представити довільну квадратну матрицю у вигляді добутку нижньої та верхньої трикутних матриць. Такий підхід широко застосовується для розв'язування систем лінійних рівнянь, обчислення обернених матриць та визначників. У практичній частині роботи було реалізовано як послідовний, так і паралельний варіанти LU-розкладу з використанням технології OpenMP. Паралелізація виконувалась за допомогою механізму задач (task), що дозволяє розподіляти обчислення між потоками та потенційно пришвидшувати виконання алгоритму.

Додаток:

```
#include <stdio.h>    // стандартний ввід/вивід (printf, scanf)
```

```

#include <stdlib.h> // для rand(), srand()
#include <omp.h>    // бібліотека для паралельних обчислень OpenMP
#include <time.h>   // для генерації випадкових чисел через час

#define MAX 1000    // максимальний розмір матриці

// глобальні матриці (щоб не передавати їх у кожен функцію)
double A[MAX][MAX]; // початкова матриця
double L[MAX][MAX]; // нижня трикутна
double U[MAX][MAX]; // верхня трикутна

// очищення матриць L і U
void reset_LU(int n)
{
    // просто проходимося по всіх елементах і зануляємо
    for(int i=0;i<n;i++)
        for(int j=0;j<n;j++)
        {
            L[i][j] = 0.0;
            U[i][j] = 0.0;
        }
}

// введення матриці вручну
void input_matrix_console(int n)
{
    printf("\nEnter matrix elements:\n");

    // користувач сам вводить кожен елемент
    for(int i=0;i<n;i++)
        for(int j=0;j<n;j++)
        {
            printf("A[%d][%d] = ",i,j);
            scanf("%lf",&A[i][j]);
        }
}

// випадкова генерація
void generate_matrix_random(int n)
{
    srand(time(NULL)); // ініціалізація генератора
    // заповнюємо числами від 1 до 10
    for(int i=0;i<n;i++)
        for(int j=0;j<n;j++)
            A[i][j] = rand()%10 + 1;
}

// генерація за функцією
void generate_matrix_function(int n)
{
    // проста формула щоб було легко перевірити правильність
    for(int i=0;i<n;i++)
        for(int j=0;j<n;j++)

```

```

        A[i][j] = i + j + 1;
    }

// друк матриці
void print_matrix(double M[MAX][MAX], int n)
{
    // вивід матриці
    for(int i=0;i<n;i++)
    {
        for(int j=0;j<n;j++)
            printf("%8.2f ",M[i][j]); // форматування

        printf("\n");
    }
}

// запис результатів у файл
void write_result(int n, double seq_time, double times[], int threads_count)
{
    FILE *f = fopen("result.txt","w"); // відкриваємо файл

    fprintf(f,"Matrix size: %d\n\n",n);

    // записуємо час послідовного алгоритму
    fprintf(f,"Sequential time: %f\n\n",seq_time);

    fprintf(f,"Parallel results:\n");

    int thread_list[4] = {1,2,4,8};

    // запис часу для різної кількості потоків
    for(int i=0;i<threads_count;i++)
        fprintf(f,"Threads %d : %f seconds\n",thread_list[i],times[i]);

    // запис матриці L
    fprintf(f,"\nMatrix L:\n");

    for(int i=0;i<n;i++)
    {
        for(int j=0;j<n;j++)
            fprintf(f,"%8.3f ",L[i][j]);

        fprintf(f,"\n");
    }

    // запис матриці U
    fprintf(f,"\nMatrix U:\n");

    for(int i=0;i<n;i++)
    {
        for(int j=0;j<n;j++)
            fprintf(f,"%8.3f ",U[i][j]);
    }
}

```

```

        fprintf(f, "\n");
    }

    fclose(f); // закриваємо файл
}

// послідовний LU-розклад (метод Дулітла)

void lu_sequential(int n)
{
    // основний цикл по рядках
    for(int i=0; i<n; i++)
    {
        // ----- обчислення U -----
        for(int k=i; k<n; k++)
        {
            double sum = 0;

            // рахуємо суму добутоків
            for(int j=0; j<i; j++)
                sum += L[i][j]*U[j][k];

            // формула для U
            U[i][k] = A[i][k] - sum;
        }

        // ----- обчислення L -----
        for(int k=i; k<n; k++)
        {
            if(i==k)
                L[i][i] = 1; // діагональ = 1
            else
            {
                double sum = 0;

                for(int j=0; j<i; j++)
                    sum += L[k][j]*U[j][i];

                // формула для L
                L[k][i] = (A[k][i] - sum)/U[i][i];
            }
        }
    }
}

// паралельний LU з використанням tasks
void lu_parallel(int n)
{
#pragma omp parallel // запускаємо паралельну область
{
#pragma omp single // тільки один потік створює задачі

```

```

    for(int i=0;i<n;i++)
    {

        // ----- U -----
        for(int k=i;k<n;k++)
        {

#pragma omp task firstprivate(i,k) // створюємо задачу
            {
                double sum = 0;

                for(int j=0;j<i;j++)
                    sum += L[i][j]*U[j][k];

                U[i][k] = A[i][k] - sum;
            }
        }

#pragma omp taskwait // чекаємо завершення задач

        // ----- L -----
        for(int k=i;k<n;k++)
        {

#pragma omp task firstprivate(i,k)
            {
                if(i==k)
                    L[i][i] = 1;
                else
                {
                    double sum = 0;

                    for(int j=0;j<i;j++)
                        sum += L[k][j]*U[j][i];

                    L[k][i] = (A[k][i] - sum)/U[i][i];
                }
            }
        }

#pragma omp taskwait // знову чекаємо

    }
}

// main
int main()
{
    int n;

```

```
int choice;

// введення розміру
printf("Enter matrix size: ");
scanf("%d",&n);

// перевірка
if(n>MAX)
{
    printf("Matrix too large!\n");
    return 0;
}

// вибір способу заповнення
printf("\nChoose input method:\n");
printf("1 - Manual input\n");
printf("2 - Random generation\n");
printf("3 - Function generation\n");
printf("Choice: ");

scanf("%d",&choice);

// виклик відповідної функції
if(choice==1)
    input_matrix_console(n);
else if(choice==2)
    generate_matrix_random(n);
else
    generate_matrix_function(n);

// показуємо матрицю
printf("\nMatrix A:\n");
print_matrix(A,n);

double start,end;

// послідовний алгоритм

reset_LU(n); // очистка

start = omp_get_wtime(); // старт часу

lu_sequential(n); // запуск

end = omp_get_wtime(); // кінець

double seq_time = end-start;

printf("\nSequential time: %f\n",seq_time);

// паралельні тести

int thread_list[4] = {1,2,4,8};
```

```
double par_times[4];

// тестуємо різну кількість потоків
for(int t=0;t<4;t++)
{
    omp_set_num_threads(thread_list[t]); // встановлюємо потоки

    reset_LU(n);

    start = omp_get_wtime();

    lu_parallel(n);

    end = omp_get_wtime();

    par_times[t] = end-start;

    printf("Threads %d : %f\n",thread_list[t],par_times[t]);
}
// запис у файл

write_result(n,seq_time,par_times,4);

printf("\nResults saved to result.txt\n");

return 0;
}
```